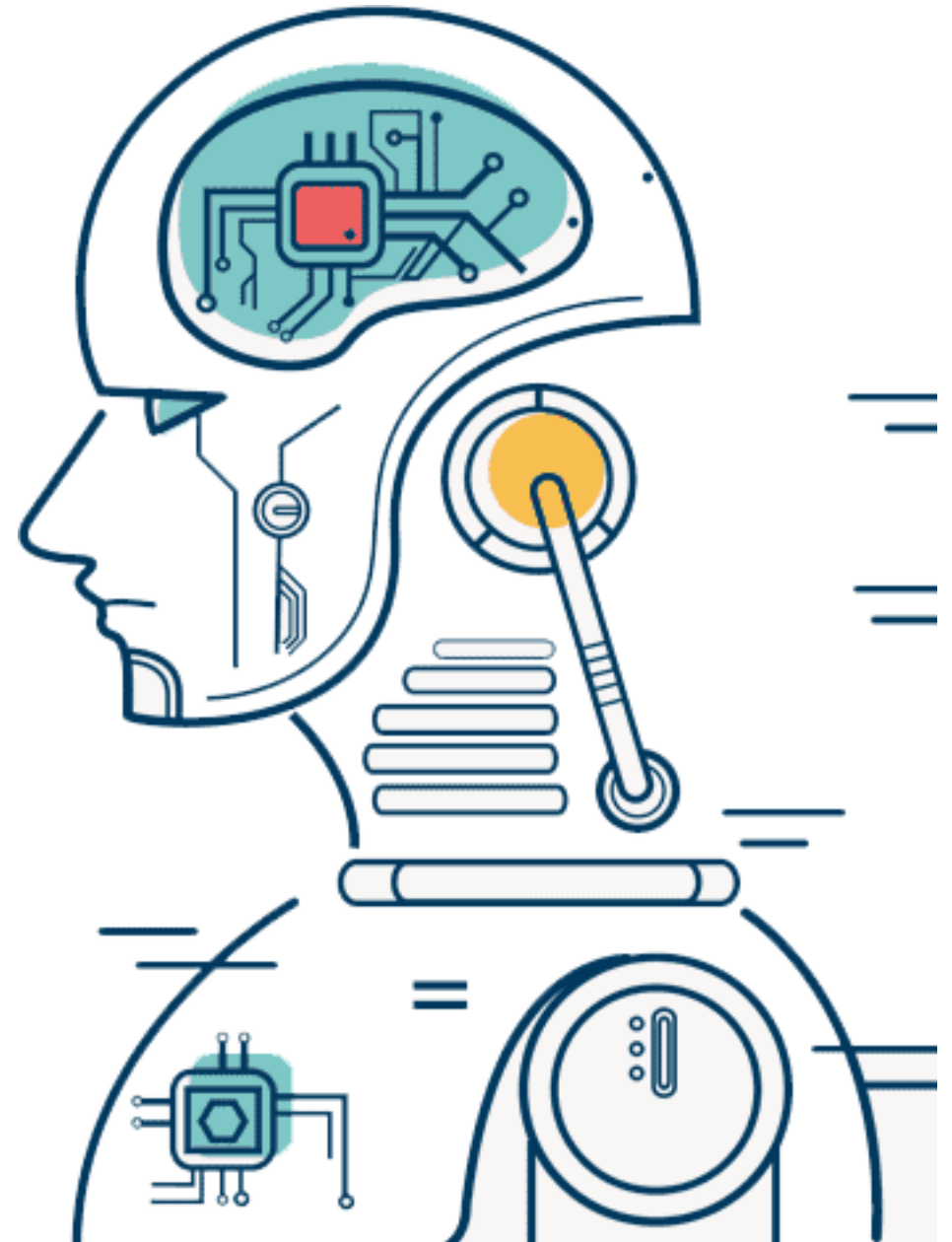


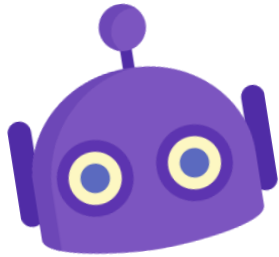
Machine Learning

Chapter 03 Decision Tree, Label Encoding, One-hot Encoding, Cross validation

양재원 연구원



- Decision Tree 알고리즘을 이해 할 수 있다.
- Label 인코딩과 One-hot 인코딩을 이해 할 수 있다.
- 정답(레이블)과 연관 관계가 높은 특성을 선택할 수 있다.
- 교차 검증 기법을 이해 할 수 있다.

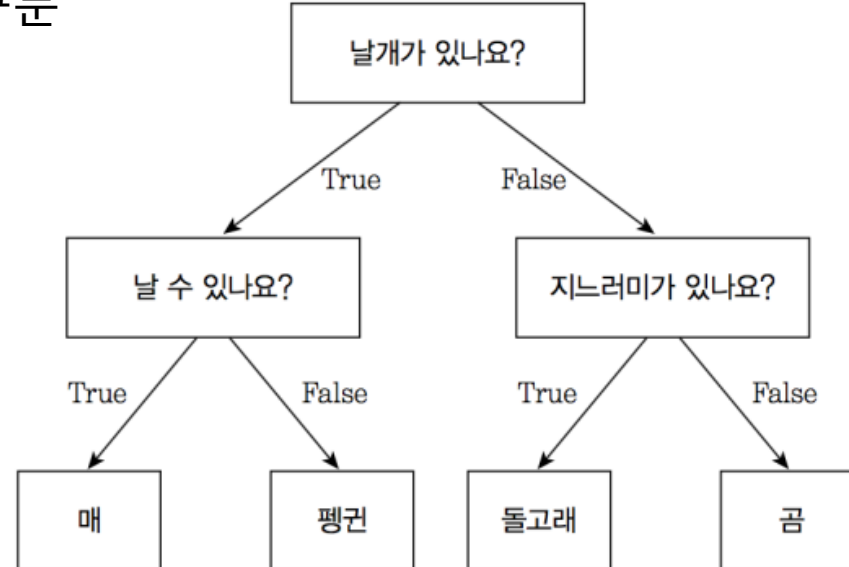


Decision Tree 의사결정나무 모델

Decision Tree

- 스무고개 하듯이 질문을 하고, 예/아니오 답변을 반복하며 학습
- 특정 기준(질문)에 따라 데이터를 구분하는 모델
- 분류와 회귀에 모두 사용 가능

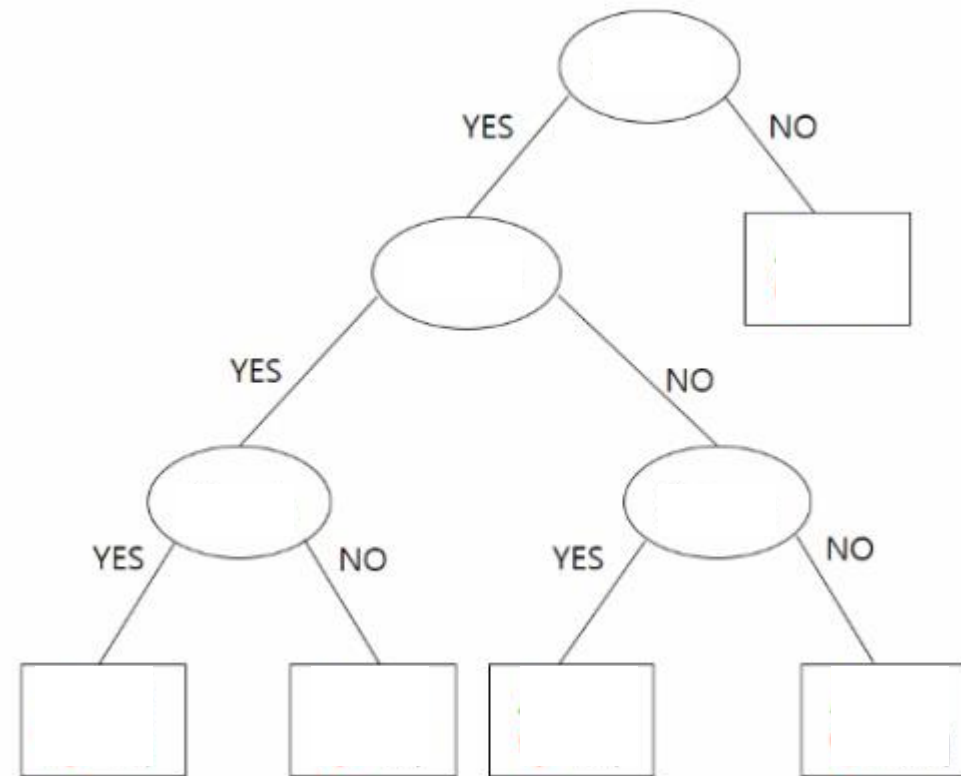
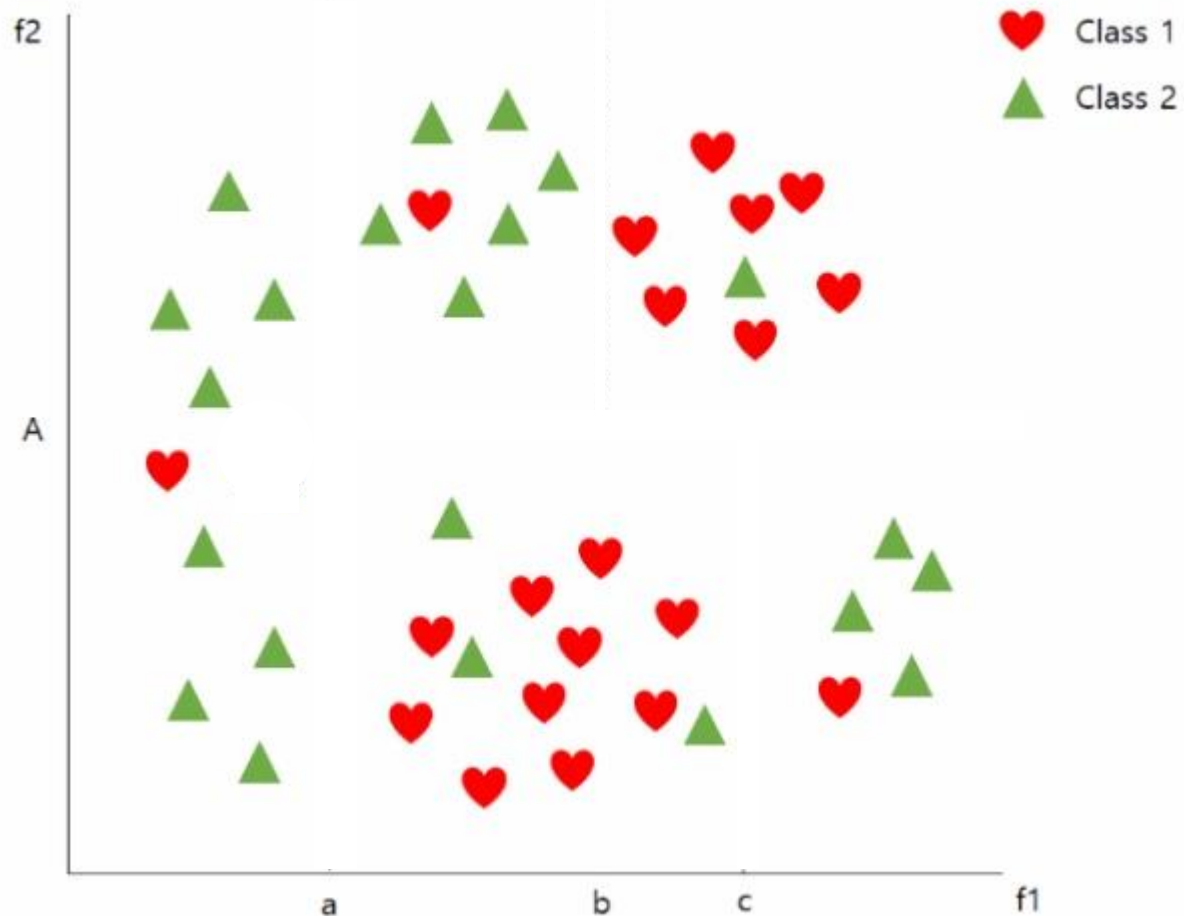
예) 매, 펭귄, 돌고래, 곰을 구분



의사결정게임



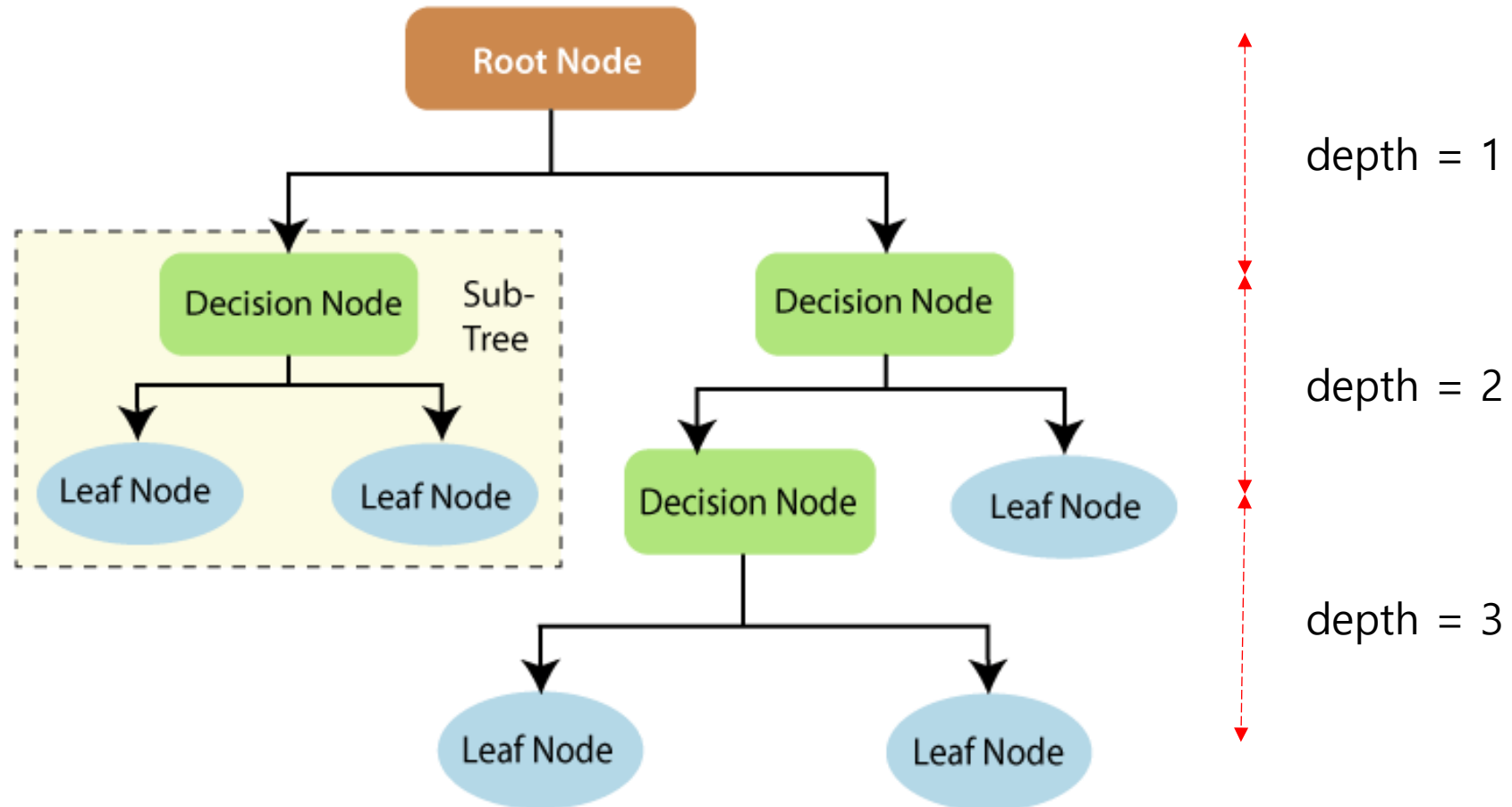
Decision Tree 의사 결정 방향 → 불순도가 낮아지는 방향



지니 불순도 (Gini Impurity)

- 해당 범주 안에 서로 다른 데이터가 얼마나 섞여 있는지를 뜻함
- 결정 트리 모델의 노드 분할 기준
- 각 질문들이 얼마나 좋은 질문인지 수치로 파악 할 수 있음
- 0 ~ 0.5 사이 값을 범위로 가짐
- 불순도가 0에 가까울수록 잘 분류된 것 (좋은 질문)
- 불순도가 0.5라면 데이터가 5:5 비율로 섞여서 분류된 것 (좋지 않은 질문)

Decision Tree Root Node, Decision Node, Leaf Node, depth, Sub Tree



Decision Tree 사용법 : 주요 매개변수(hyperparameter)

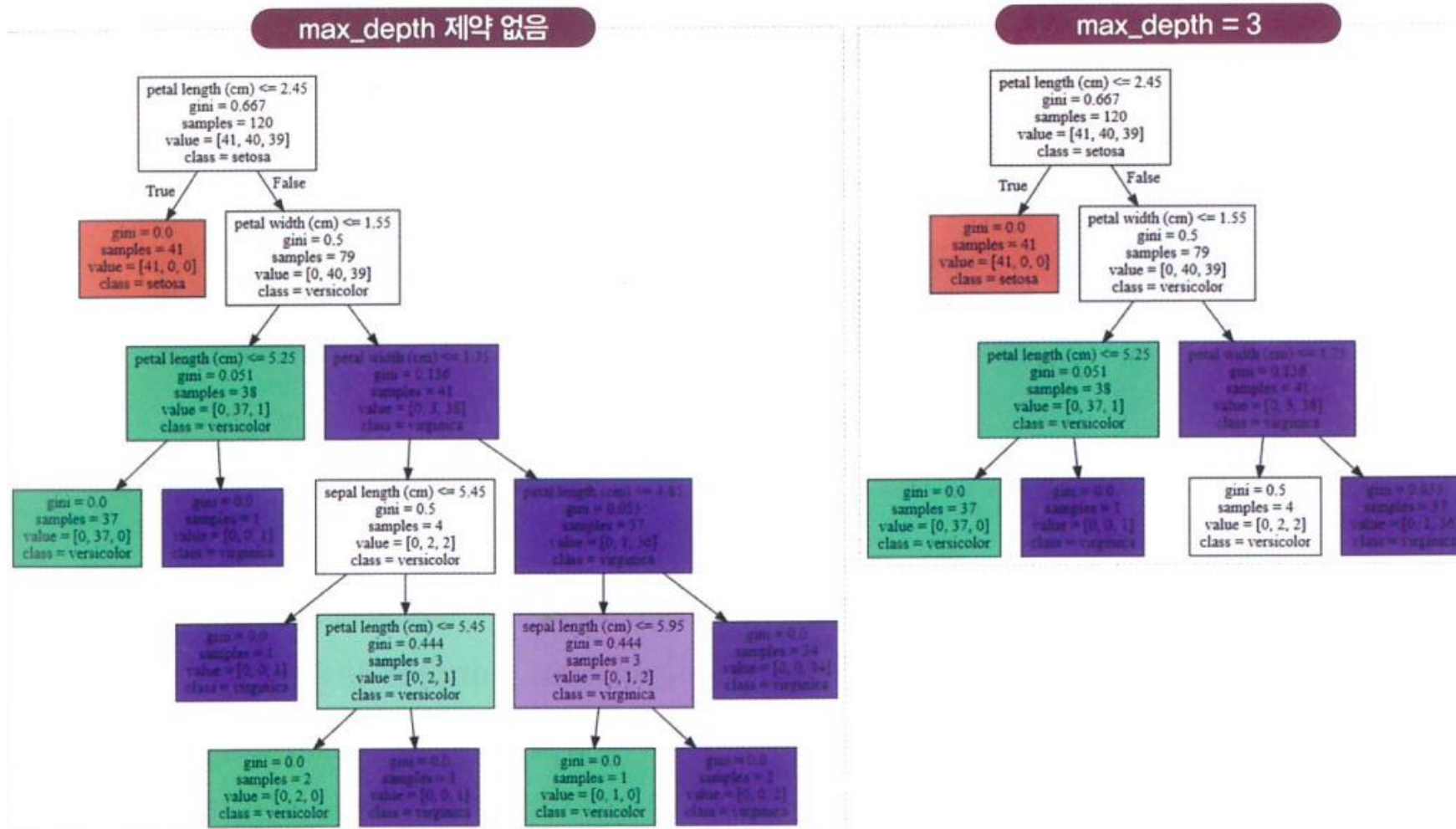
```
DecisionTreeClassifier(criterion, max_depth, min_samples_split,  
                        min_samples_leaf, max_leaf_nodes)
```

- **criterion** : 기준/불순도 측정 방법 (gini)
- **max_depth** : 트리의 최대 깊이
- **min_samples_split** : 노드를 분할하기 위한 최소 샘플 수
- **min_samples_leaf** : 리프 노드가 가져야할 최소 샘플 수
- **max_leaf_nodes** : 리프 노드의 최대 개수

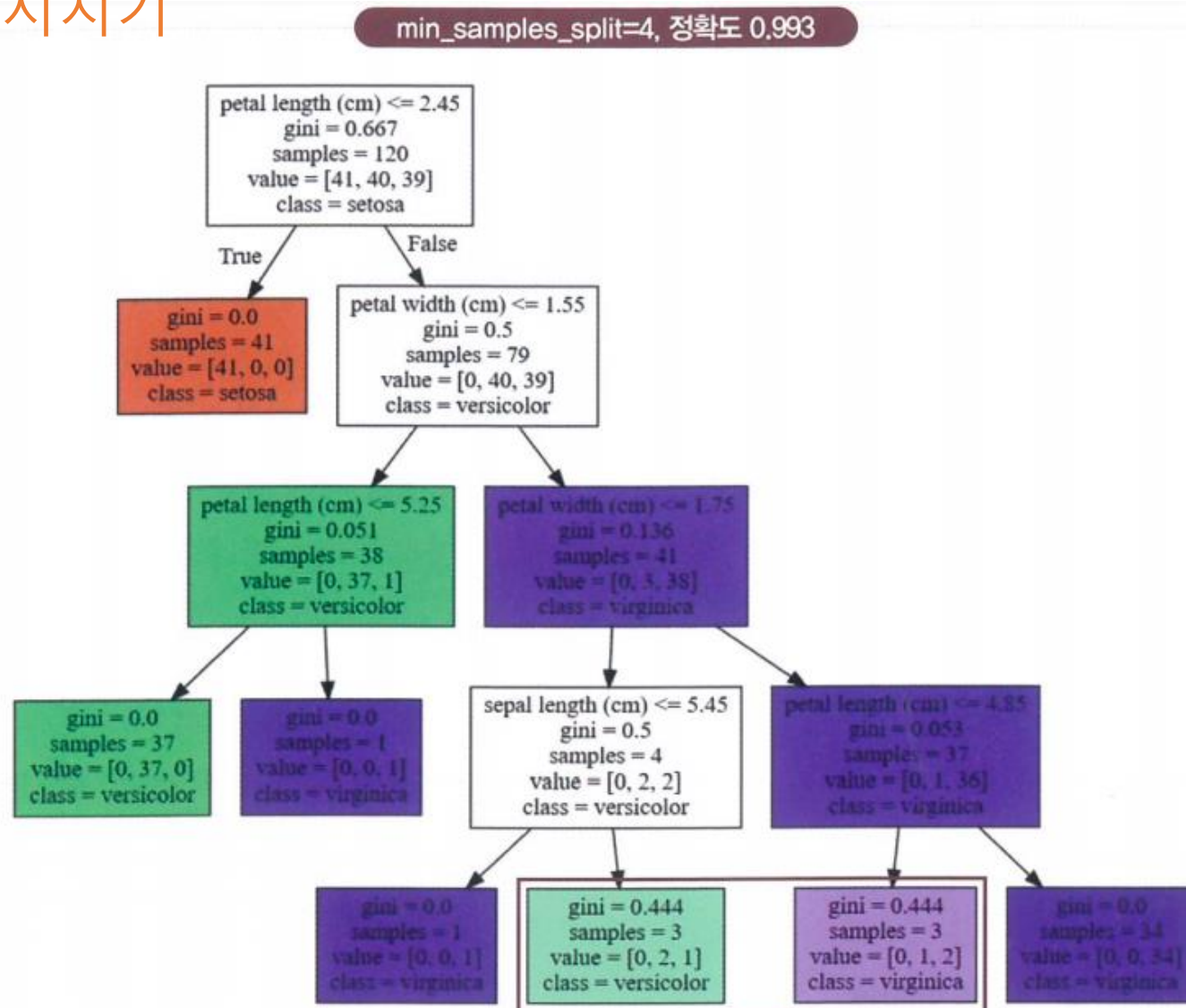
Decision Tree 과대적합 제어

- **max_depth** : 트리의 최대 깊이, **값이 클수록 모델의 복잡도가 올라간다.**
- **max_leaf_nodes** : 리프 노드의 최대 개수, **크게 설정될수록 분할되는 노드가 많아져서 과대적합 가능성 증가**
- **min_samples_split** : 노드를 분할하기 위한 최소 샘플 수, **작게 설정될수록 분할되는 노드가 많아져서 과대적합 가능성 증가**
- **min_samples_leaf** : 리프 노드가 가져야할 최소 샘플 수, **작게 설정될수록 분할되는 노드가 많아져서 과대적합 가능성 증가**

Decision Tree 사전 가지치기

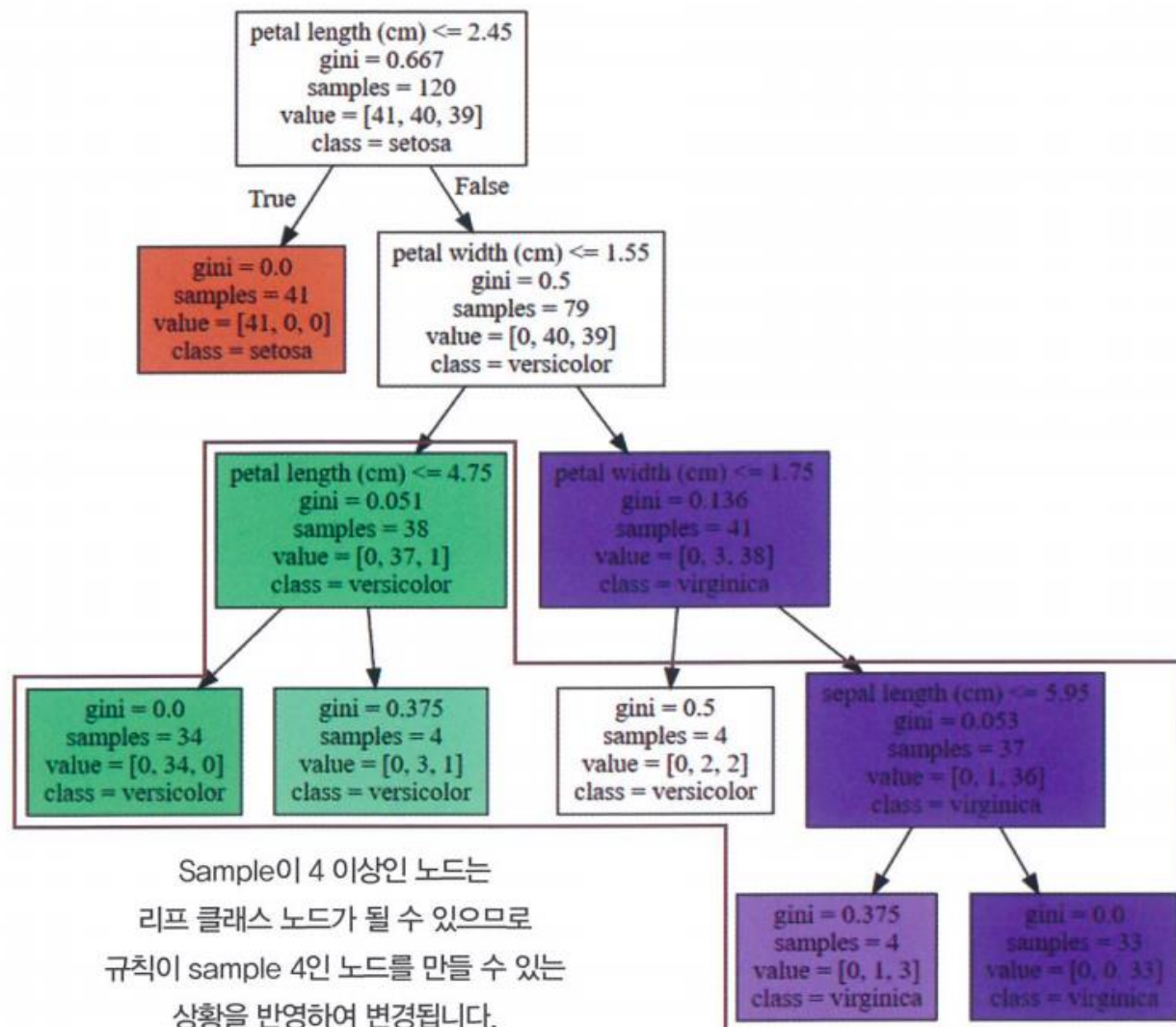


Decision Tree 사전 가지치기



Decision Tree 사전 가지치기

min_samples_leaf=4, 정확도 0.933



Decision Tree 장단점

장점	단점
• 쉽다 • 직관적이다	• 과대적합이 발생하기 쉬움 • 이를 극복하기 위해서 트리의 크기를 사전에 제한하는 튜닝이 필요 (사전 가지치기)

Decision Tree 실습

Mushroom 데이터 활용 Decision Tree 분류 실습

Decision Tree 사용법

필요한 라이브러리 import

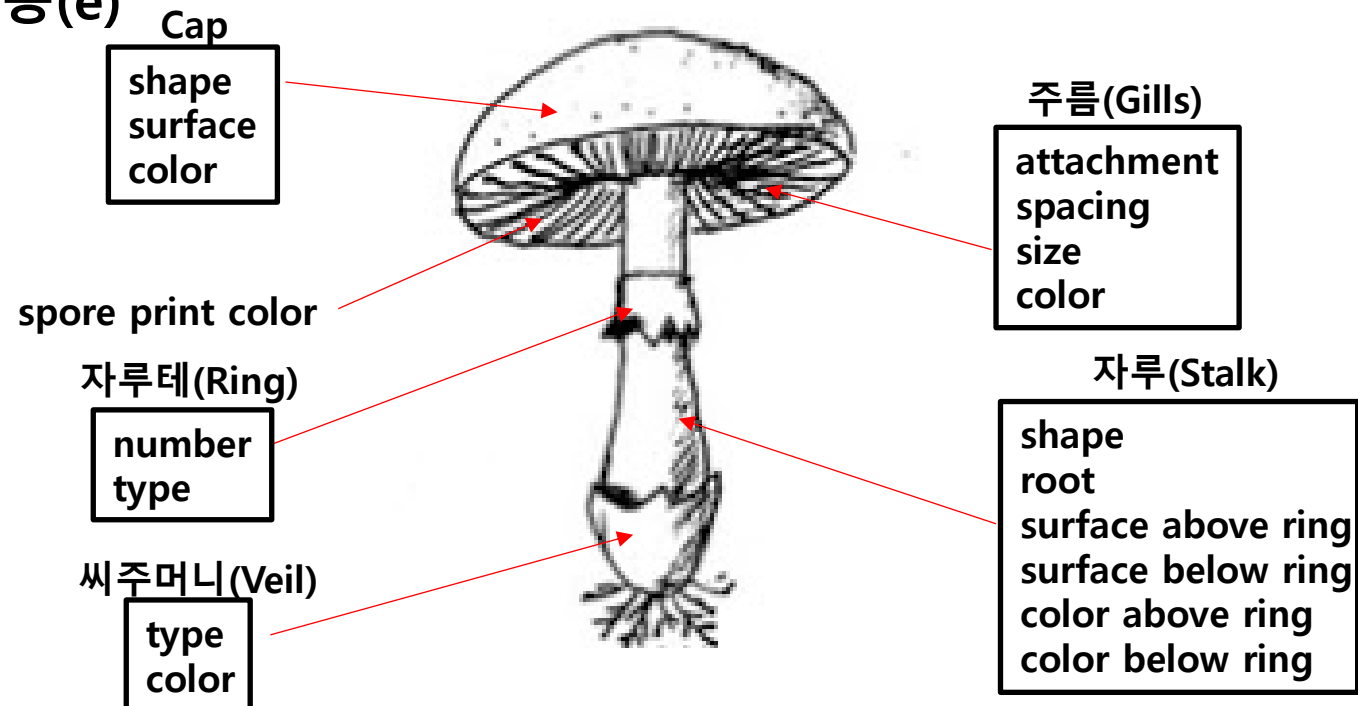
from sklearn.tree import DecisionTreeClassifier

결정 트리 분류 모델 생성 함수 호출, 결정 트리 분류 모델 객체 생성

tree_model = DecisionTreeClassifier(하이퍼 파라미터, random_state)

Mushroom 데이터 셋

- 8124개의 버섯 종류 데이터
- 22개의 특징 (18개의 버섯 특징, 4개의 다른 특징 (Habitat (서식지), Population(분포 형태), Bruises(타박상), Odor(냄새)))
- 라벨 : 독성 (p), 식용(e)



Mushroom 데이터 셋

poisonous : 독버섯(poisonous), 식용버섯(edible)
cap-shape : 갓 모양(b,c,x,f,k,s) : 원뿔/평면/볼록 등
cap-surface : 갓 표면(f,g,u,s) : 섬유질/비늘모양/부드러움 등
cap-color : 갓 색(n,b,c,g,r,p,u,e,w,y) : 계피/회색/노란색 등
bruises : 타박상(t,f) : 예/아니오
odor : 냄새(a,l,c,y,f,m,n,p,s) : 아몬드,생선,매운 등

gill-attachment(자실층 위치), **gill-spacing**(자실층 간격), **gill-size**(자실층 크기), **gill-color**(자실층 색), **stalk-shape**(자루 모양), **stalk-root**(자루 뿌리), **stalk-surface-above-ring**(자루 표면 위 자루테), **stalk-surface-below-ring**(자루 표면 아래 자루테), **stalk-color-above-ring**(자루 색 위 자루테), **stalk-color-below-ring**(자루 색 아래 자루테), **veil-type**(베일 유형), **veil-color**(베일 색), **ring-number**(링 번호), **ring-type**(링 타입), **spore-print-color**(포자 색), **population**(인구), **habitat**(서식지)

함수1. info() 함수 - 결측치 확인

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
poisonous      8124 non-null object
cap-shape      8124 non-null object
cap-surface    8124 non-null object
cap-color      8124 non-null object
bruises        8124 non-null object
odor           8124 non-null object
gill-attachment 8124 non-null object
gill-spacing   8124 non-null object
gill-size      8124 non-null object
gill-color     8124 non-null object
stalk-shape    8124 non-null object
stalk-root     8124 non-null object
stalk-surface-above-ring 8124 non-null object
stalk-surface-below-ring 8124 non-null object
stalk-color-above-ring 8124 non-null object
stalk-color-below-ring 8124 non-null object
veil-type      8124 non-null object
veil-color     8124 non-null object
ring-number    8124 non-null object
ring-type      8124 non-null object
spore-print-color 8124 non-null object
population     8124 non-null object
habitat        8124 non-null object
dtypes: object(23)
memory usage: 1.4+ MB
```

함수2. value_counts() 함수 – 데이터의 종류와 개수 확인

`X['cap-shape'].value_counts()`

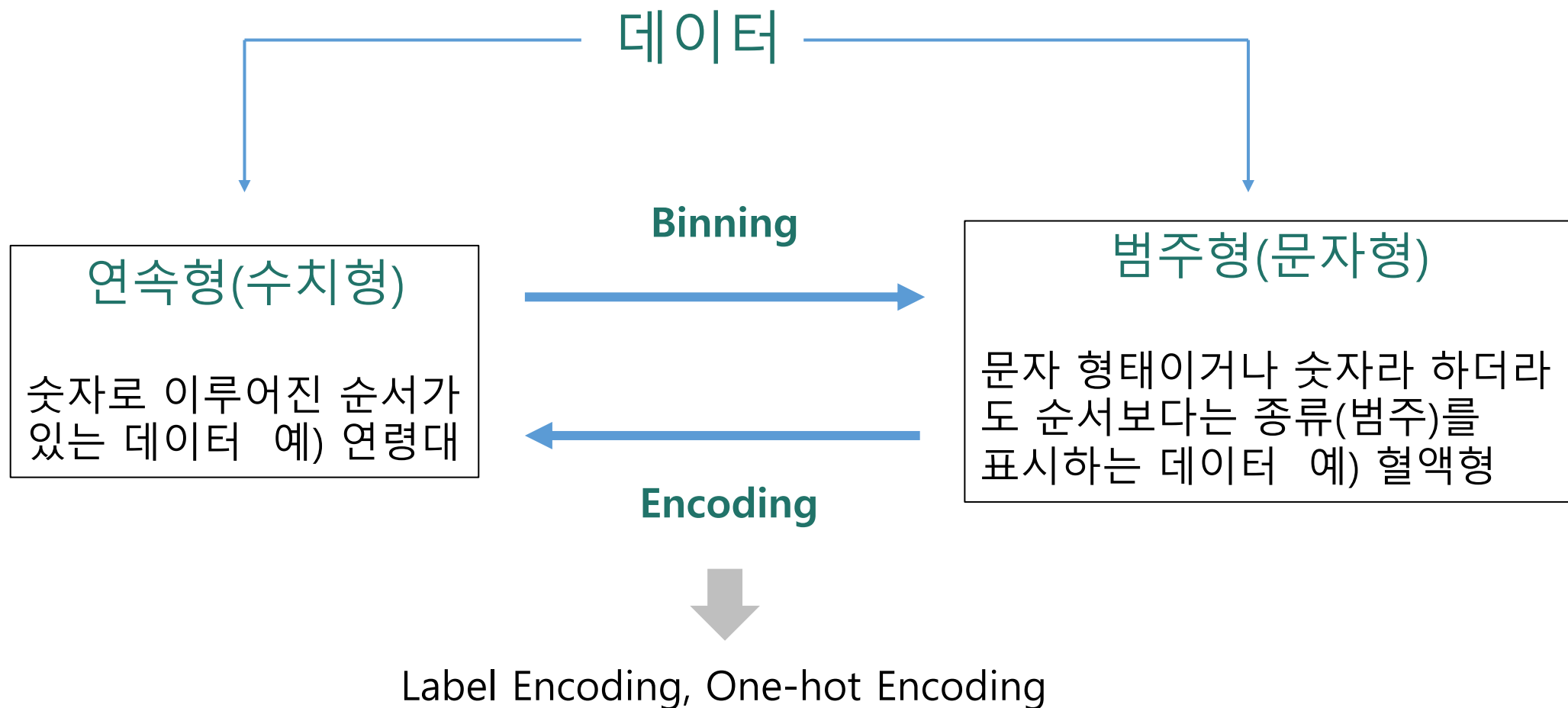
```
x      3656
f      3152
k       828
b       452
s        32
c         4
Name: cap-shape, dtype: int64
```

함수3. unique() 함수 – 데이터의 종류 확인

`X['cap-shape'].unique()`

```
['x' 'b' 's' 'f' 'k' 'c']
```

데이터 전처리 : 인코딩(Encoding)



데이터 표현

- **연속형(수치형) 특성** : 숫자로 이루어진 순서가 있는 데이터
- **범주형(문자형) 특성** : 문자 형태로 된 값으로 이루어진 데이터
- **Encoding** : 범주형 데이터를 숫자형 데이터로 변환
(Label Encoding, One-hot Encoding 등)
- **Binning** : 숫자형 데이터를 범주형 데이터로 변환

Mushroom 데이터셋 : 범주형(문자형) 데이터, 인코딩 필요

	class	cap- shape	cap- surface	cap- color	bruises	odor	gill- attachment	gill- spacing	gill- size	gill- color	...	stalk- surface- below- ring	stalk- color- above- ring	stalk- color- below- ring	veil- type	veil- color
0	p	x	s	n	t	p	f	c	n	k	...	s	w	w	p	w
1	e	x	s	y	t	a	f	c	b	k	...	s	w	w	p	w
2	e	b	s	w	t	l	f	c	b	n	...	s	w	w	p	w
3	p	x	y	w	t	p	f	c	n	n	...	s	w	w	p	w
4	e	x	s	g	f	n	f	w	b	k	...	s	w	w	p	w



One-hot 인코딩 방식을 이용해 수치화

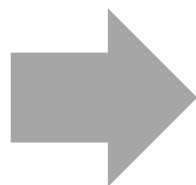
One-hot 인코딩 방식을 이용해 수치화

	cap- shape_b	cap- shape_c	cap- shape_f	cap- shape_k	cap- shape_s	cap- shape_x
0	0	0	0	0	0	1
1	0	0	0	0	0	1
2	1	0	0	0	0	0
3	0	0	0	0	0	1
4	0	0	0	0	0	1

데이터 전처리 One-hot Encoding : 분류하고자 하는 범주(종류) 만큼의 자릿수를 만들고 단 한 개의 1과 나머지 0으로 채워서 숫자화 하는 방식

`X_one_hot = pd.get_dummies(X2)`

cap-shape
x
b
x
k
f
s
b
s
c



cap-shape_b	cap-shape_c	cap-shape_f	cap-shape_k	cap-shape_s	cap-shape_x
0	0	0	0	0	1
1	0	0	0	0	0
0	0	0	0	0	1
0	0	0	1	0	0
0	0	1	0	0	0
0	0	0	0	1	0
1	0	0	0	0	0
0	0	0	0	1	0
0	1	0	0	0	0

데이터 전처리 Label Encoding : 레이블을 숫자로 mapping

```
X1["capshape"] = X1["capshape"].map({"x":0, "f":1, "k":2, "b":3, "s":4, "c":5})
```

cap-shape
x
b
x
k
f
s
b
s
c



cap-shape
0
3
0
2
1
4
3
4
5

결정 트리 분류 모델 생성

train_test_split() 함수를 이용, 데이터를 학습용과 테스트용으로 분할

결정트리 분류 모델 생성 함수 호출, 하이퍼 파라미터는 전부 기본값 설정,
분류 모델 객체 생성

```
tree_model = DecisionTreeClassifier()
```

분류 모델 학습 진행

```
tree_model.fit(X_train, y_train)
```

결정 트리 분류 모델 생성

테스트 데이터를 이용해서 예측

```
y_pred = tree_model.predict(X_test)
```

테스트 데이터에 대한 분류 모델의 성능(평균 정확도) 확인

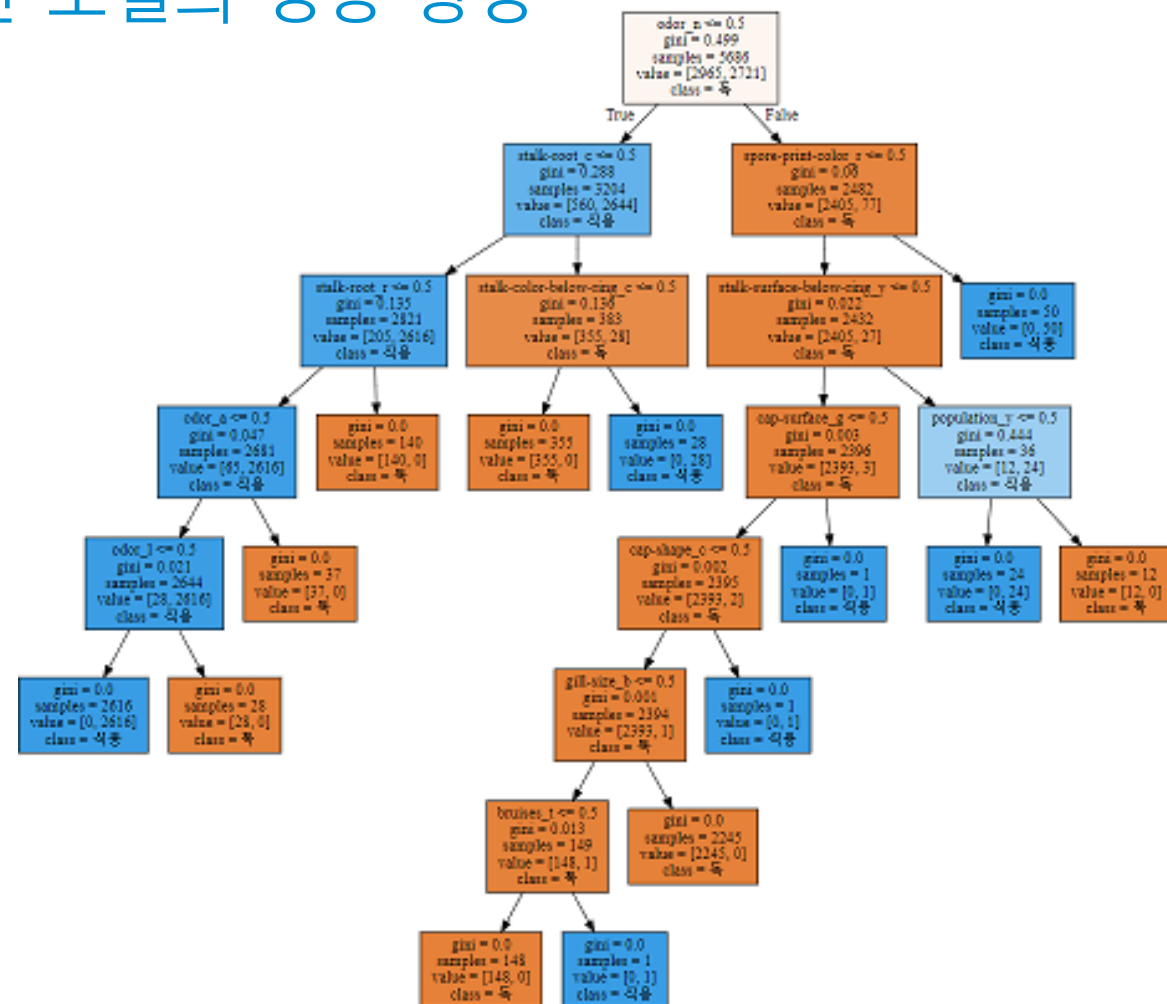
```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(accuracy)
```

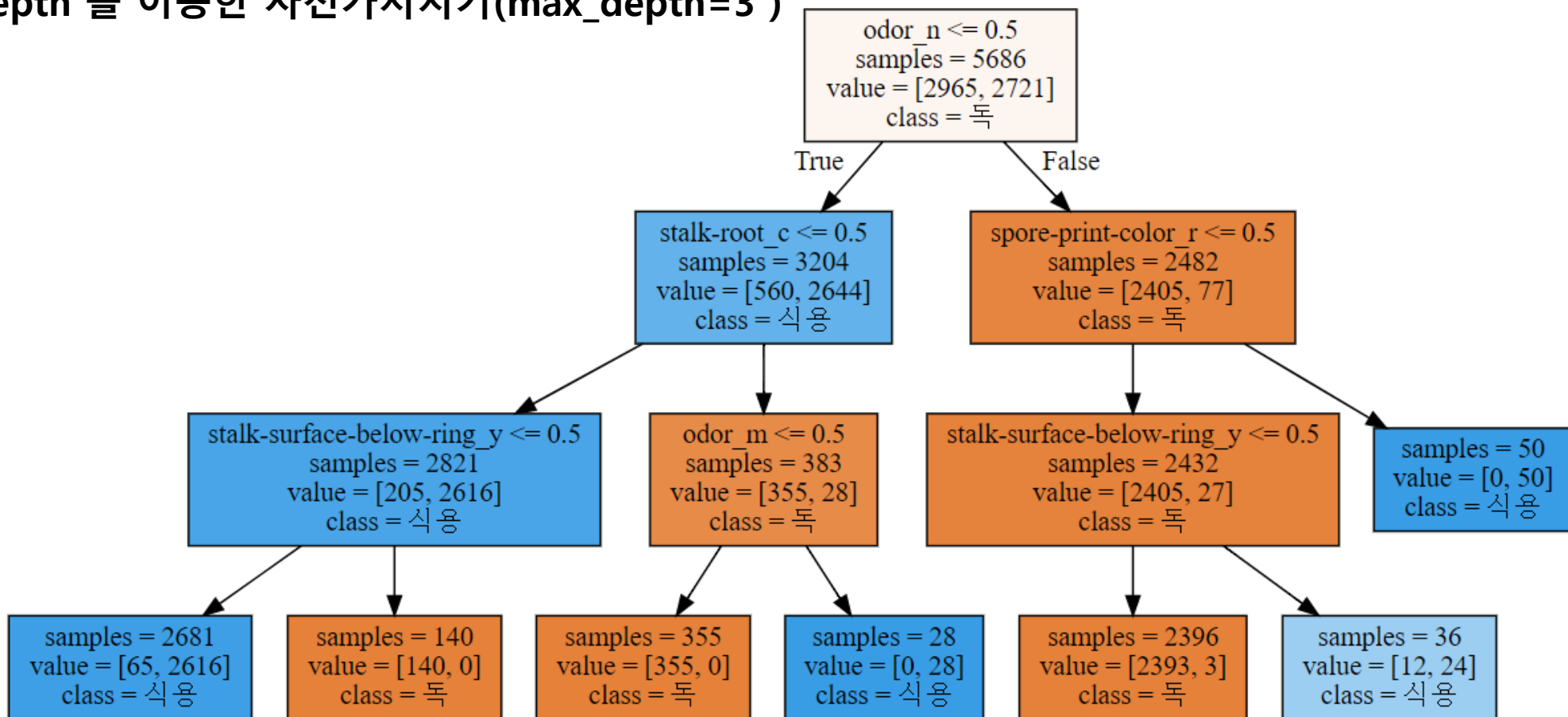
사전 가지치기(pre-pruning)를 이용한 모델의 성능 향상

- 사전 가지치기를 하지 않은 기본 모델 (tree_model)



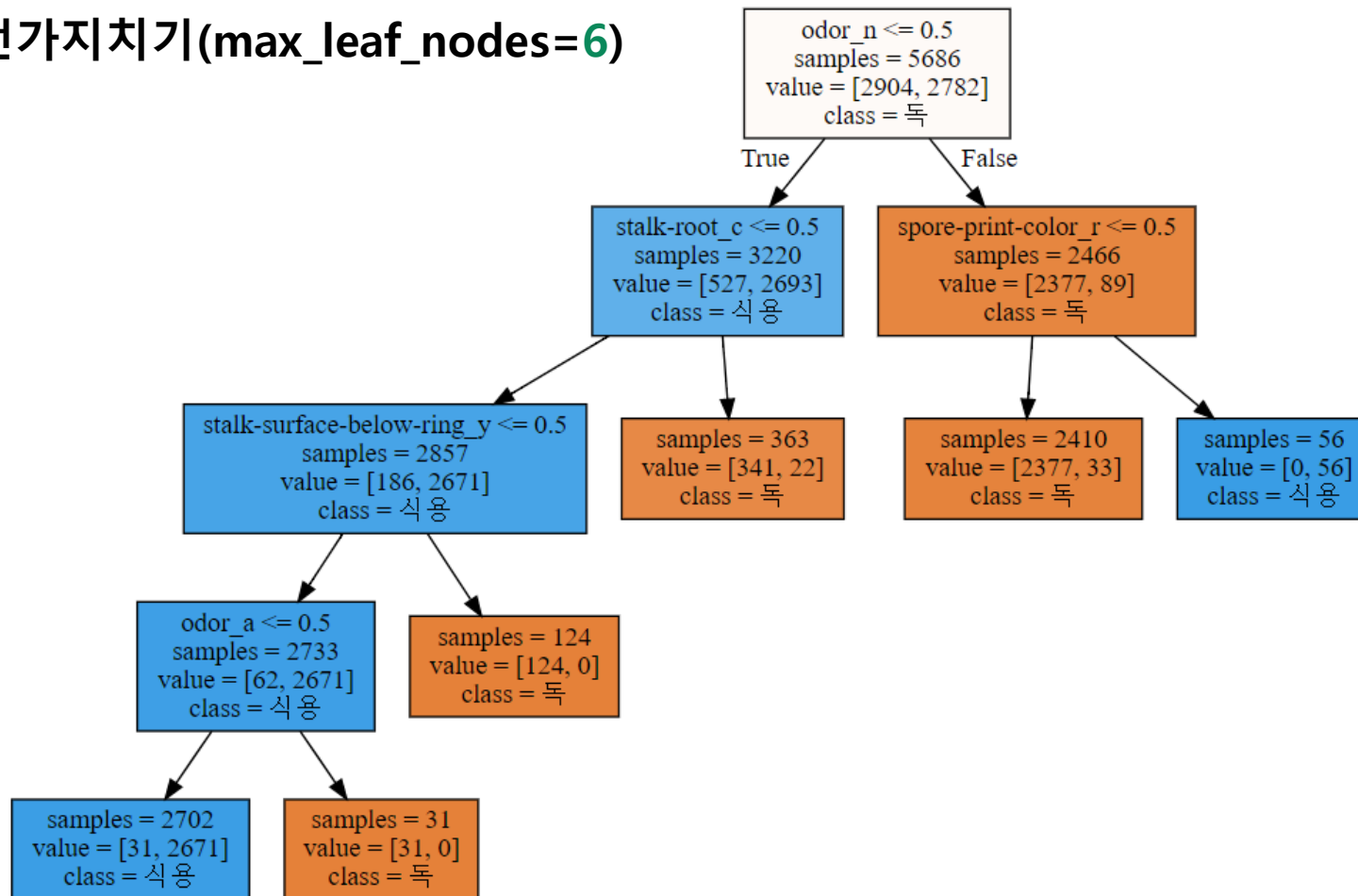
사전 가지치기(pre-pruning)를 이용한 모델의 성능 향상

- max_depth 를 이용한 사전가지치기(max_depth=3)



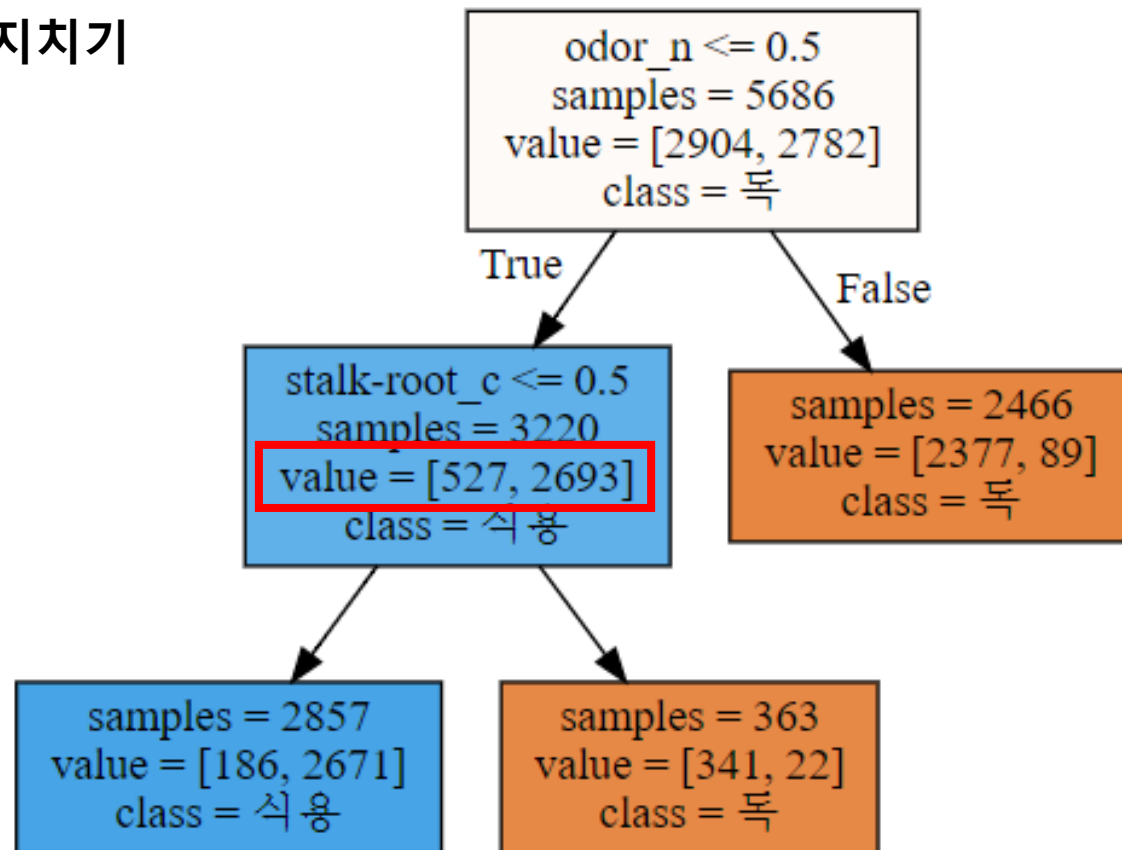
사전 가지치기(pre-pruning)를 이용한 모델의 성능 향상

- max_leaf_nodes를 이용한 사전가지치기(max_leaf_nodes=6)



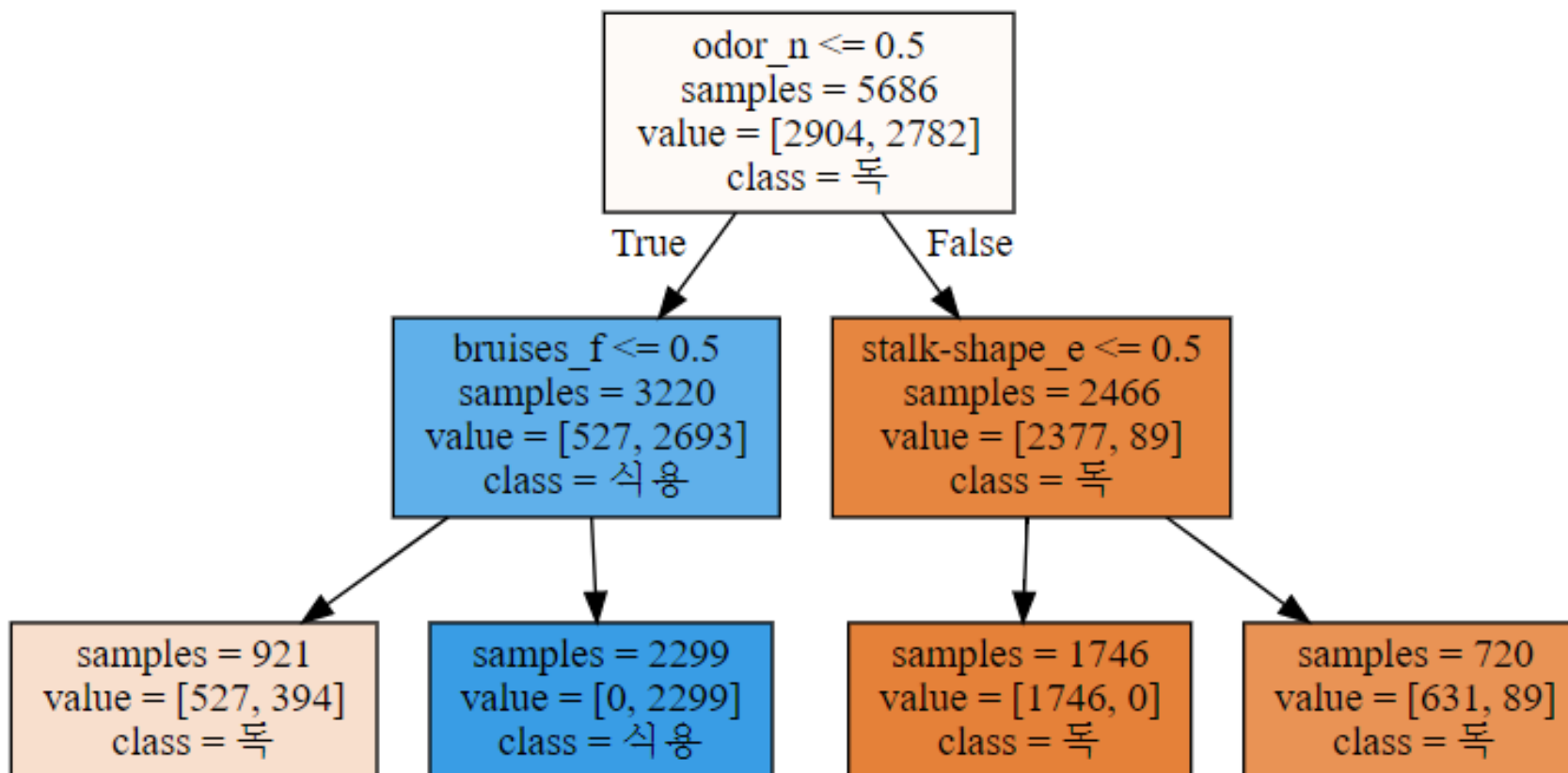
사전 가지치기(pre-pruning)를 이용한 모델의 성능 향상

- `min_samples_split`를 이용한 사전가지치기
(`min_samples_split=3000`)



사전 가지치기(pre-pruning)를 이용한 모델의 성능 향상

- min_samples_leaf를 이용한 사전가지치기(min_samples_leaf=700)

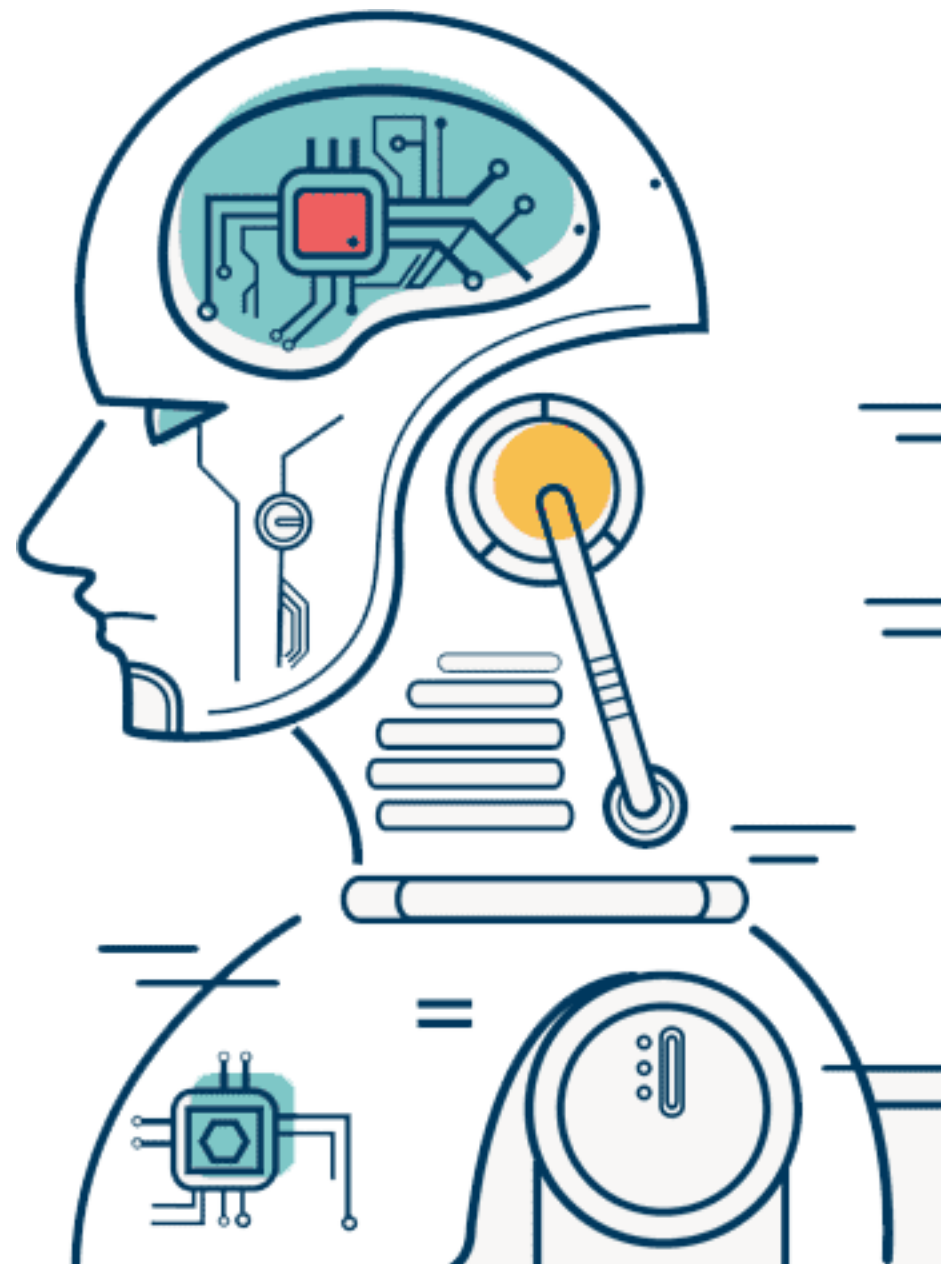


특성 선택 : 정답(레이블)과 연관 관계가 높은 특성을 선택하는 작업

```
importance = tree_model.feature_importances_  
df = pd.DataFrame(importance, index=X_one_hot.columns)  
df.sort_values(by='name', ascending=False)
```

odor_n	0.611853
stalk-root_c	0.177321
stalk-root_r	0.089846
spore-print-color_r	0.033995
odor_a	0.023077
odor_l	0.022260
stalk-surface-below-ring_y	0.017375
stalk-surface-above-ring_k	0.013956
ring-number_o	0.005403
cap-surface_g	0.002102

모델 일반화 성능 평가



학습-평가 데이터를 골고루 설정하여
모델의 안정성을 높이고 과대적합을
감소시키는 통계적 기법

모델 성능 평가 -> 교차검증

▶ **해결책** : 검증 Data를 하나로 고정하지 않고, Test 데이터의 모든 부분을 사용

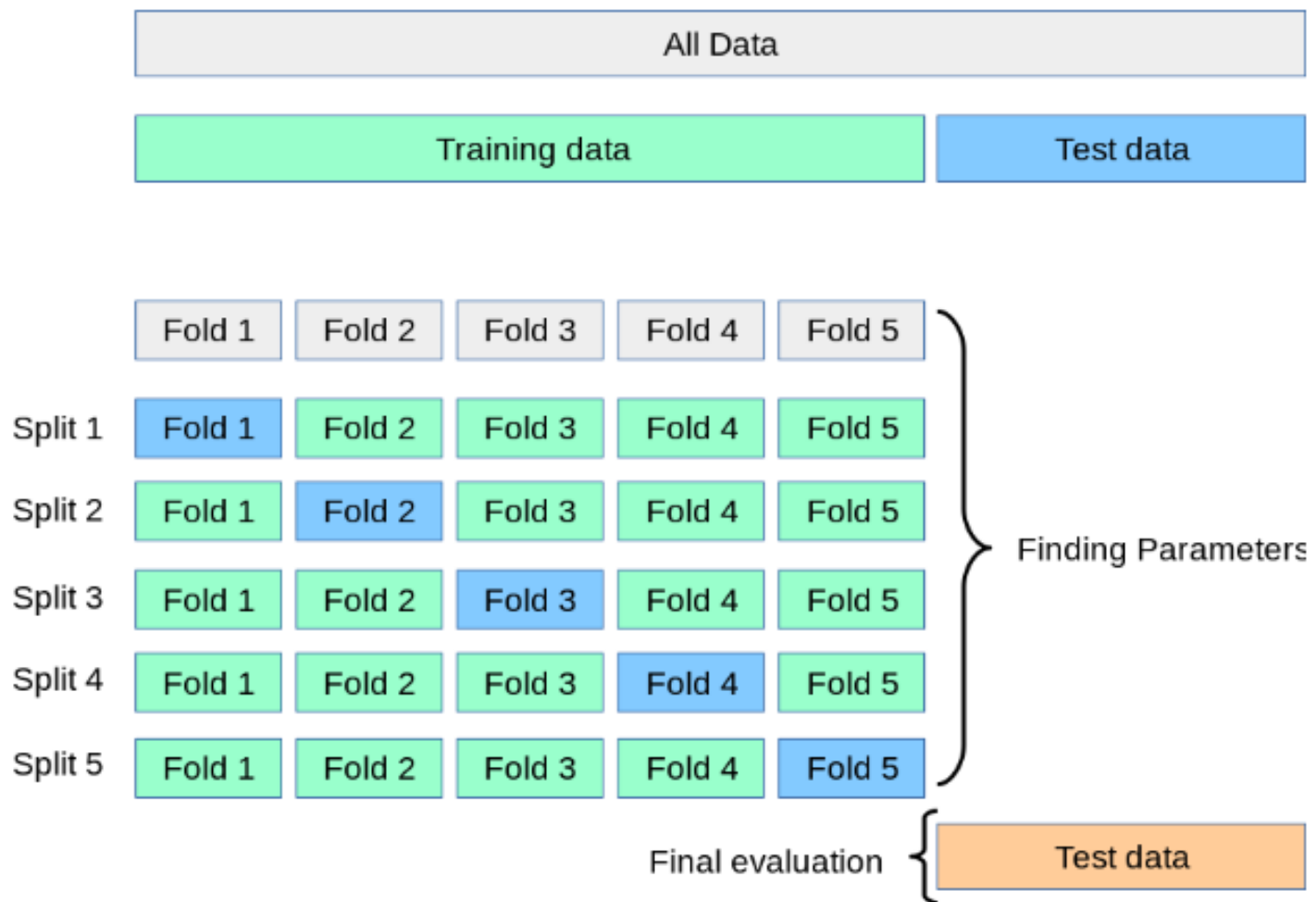


모델 성능 평가

▶ K-fold cross-validation



1. Train 데이터를 k개의 그룹으로 나누기
2. K-1개의 그룹을 학습에 사용
3. 나머지 1개의 그룹을 이용해서 평가 수행
4. 2번, 3번 과정을 k번 반복
5. 모든 결과의 평균을 구하기



K-fold cross-validation 동작 방법 ($K = 5$)



Cross-validation 장단점

- 모든 데이터 셋을 학습과 평가에 활용하기 때문에 안정적이고 정확함
→ 통계적 기법으로 과대적합을 감소시킴(일반화에 도움이 됨)
- 모델이 훈련 데이터의 변경에 대해 얼마나 민감한지 파악 가능
- 데이터 셋의 크기가 충분히 크지 않은 경우에도 유용하게 사용 가능
- 여러 번 학습하고 평가하는 과정을 거치기 때문에 계산량이 많아짐

모델 성능 평가

▶ **K-fold cross-validation 사용법** : `cross_val_score()` 함수

```
from sklearn.model_selection import cross_val_score
```

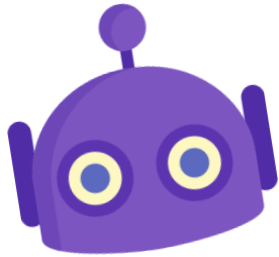
```
score = cross_val_score(모델, X, y, cv=나눌 개수)
```

```
score
```

```
[0.86680328 0.87704918 0.875          0.90554415 0.8788501 ]
```

Decision Tree

Decision Tree를 활용해 Mushroom 데이터를 학습하고 교차검증을 적용해보자.



K-Nearest Neighbors (KNN) 심화

심화 학습

KNN을 더 깊이 — 스케일링·거리·차원의 저주·학습곡선

심화 학습

KNN을 더 깊이 — 스케일링·거리·차원의 저주·학습곡선

심화 학습에서 다룰 4가지

기본 실습을 마친 뒤 진행합니다

이미 배운 것

- KNN = 가까운 이웃 k개의 다수결
- k가 작으면 과대적합, 크면 과소적합
- 거리 기반이라 스케일링이 필요하다

오늘 더 깊이

- ① 스케일러가 3종류나 있다 (뭘 쓰지?)
- ② 거리에도 종류가 있고, 가중치도 줄 수 있다
- ③ 특성이 많으면 왜 나빠지나 (차원의 저주)
- ④ '데이터를 더 모아야 하나?' 판단하는 법

① 스케일링 — 붓꽃에서는 효과가 없었는데

데이터마다 스케일링의 효과가 다르다

iris에서 관찰한 것

- 스케일링 전 0.978 → 후 0.956
- 오히려 살짝 떨어지기도 했다
- '그럼 왜 하는 거지?' 라는 의문

이유는 데이터에 있다

- iris의 4개 특성은 모두 cm 단위 (1~8 범위)
- 크기가 비슷하니 거리를 왜곡하지 않는다
→ 단위 차이가 큰 데이터로 확인해야 한다

① 와인 데이터 — 단위 차이가 극심하다

같은 KNN인데 결과가 완전히 달라지는 이유

특성	평균	최댓값	단위 성격
proline	746.9	1680.0	수백 ~ 수천
magnesium	99.7	162.0	수십 ~ 수백
alcohol	13.0	14.8	10 근처
flavanoids	2.03	5.08	1 ~ 5

거리를 그냥 계산하면 *proline*(수천 단위)이 나머지를 전부 잡아먹는다 → 다른 특성은 무시되는 셈

① 스케일러 3종 — 언제 무엇을 쓰나

상황에 따라 골라 쓴다

StandardScaler

평균 0, 표준편차 1로 변환
가장 일반적인 기본 선택

MinMaxScaler

0~1 사이로 압축
범위가 정해진 데이터에 적합

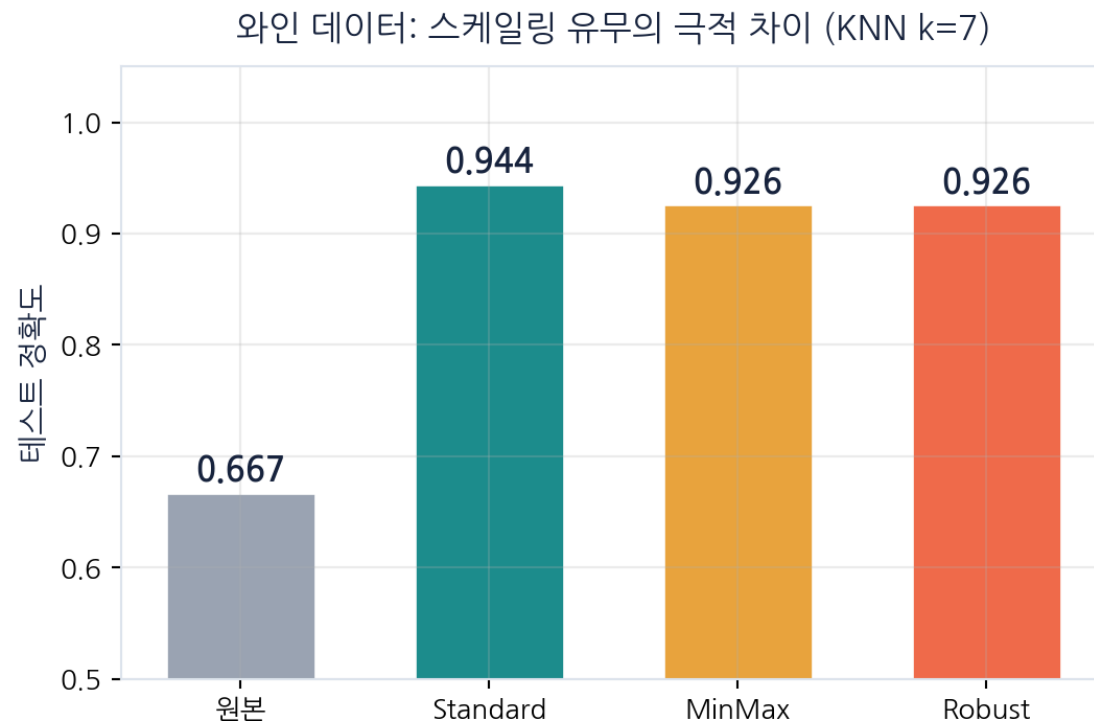
RobustScaler

중앙값·사분위수 기준
이상치가 있을 때 안정적

이상치가 섞이면 Standard·MinMax는 흔들린다. Robust는 중앙값을 쓰므로 덜 휘둘린다.

① 결과 — 정확도 0.667 → 0.944

모델도 k도 그대로, 전처리 하나로 바뀐 결과

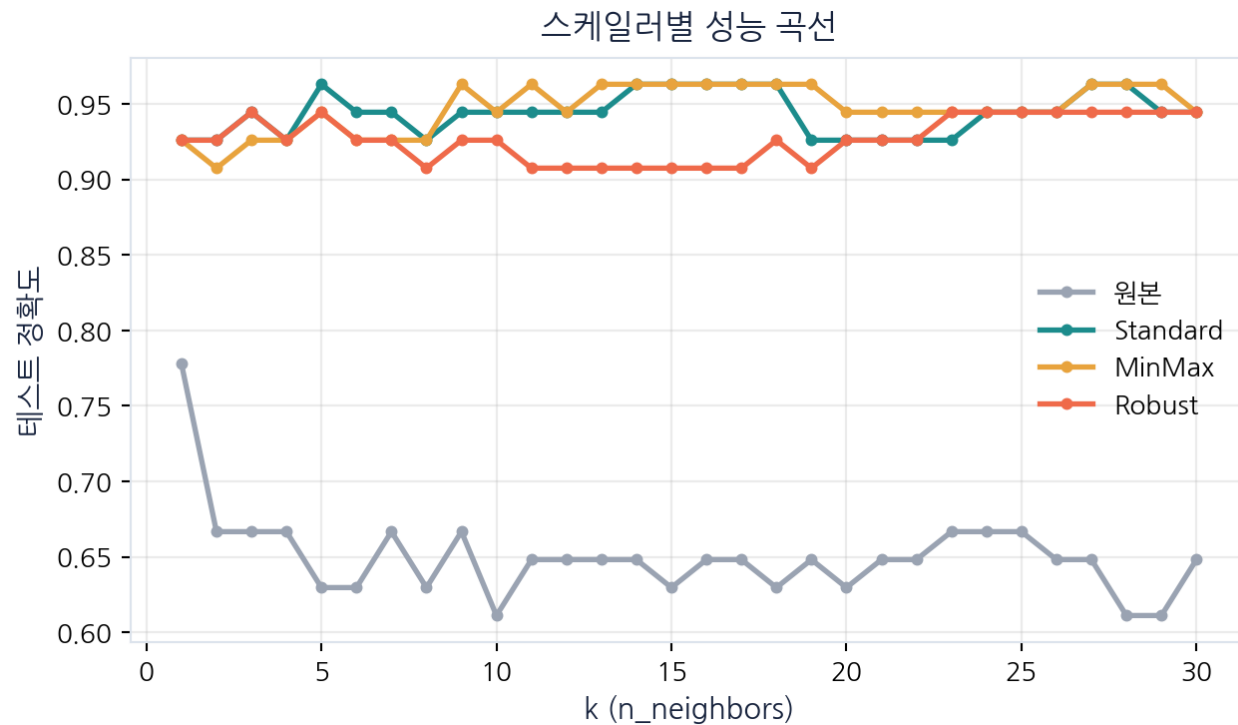


무엇을 확인했나

- 스케일링만으로 정확도 +0.28
- 모델도 k도 바꾸지 않았다
- iris는 효과 없고 wine은 극적
→ '데이터를 보고 판단하라'

① 모든 k에서 확인하면

한 지점의 우연이 아니다



읽는 법

- 회색(원본)이 계속 아래에 있다
- 특정 k에서만 좋은 게 아니다
- 스케일러 3종은 서로 비슷하다
- 핵심은 '하느냐 마느냐'

① 코드 — fit은 반드시 훈련 데이터에만

가장 흔한 실수를 피하는 법

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)          # 기준 학습
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test) # 같은 기준
```

반드시 지킬 것

- test에 fit하면 데이터 누수(반칙)
- train의 기준을 test에도 그대로
- fit_transform = fit + transform
- train에만 fit_transform 사용

② KNN의 다른 변수 — k만 있는 게 아니다

metric과 weights도 성능을 바꾼다

metric — 거리를 재는 방법

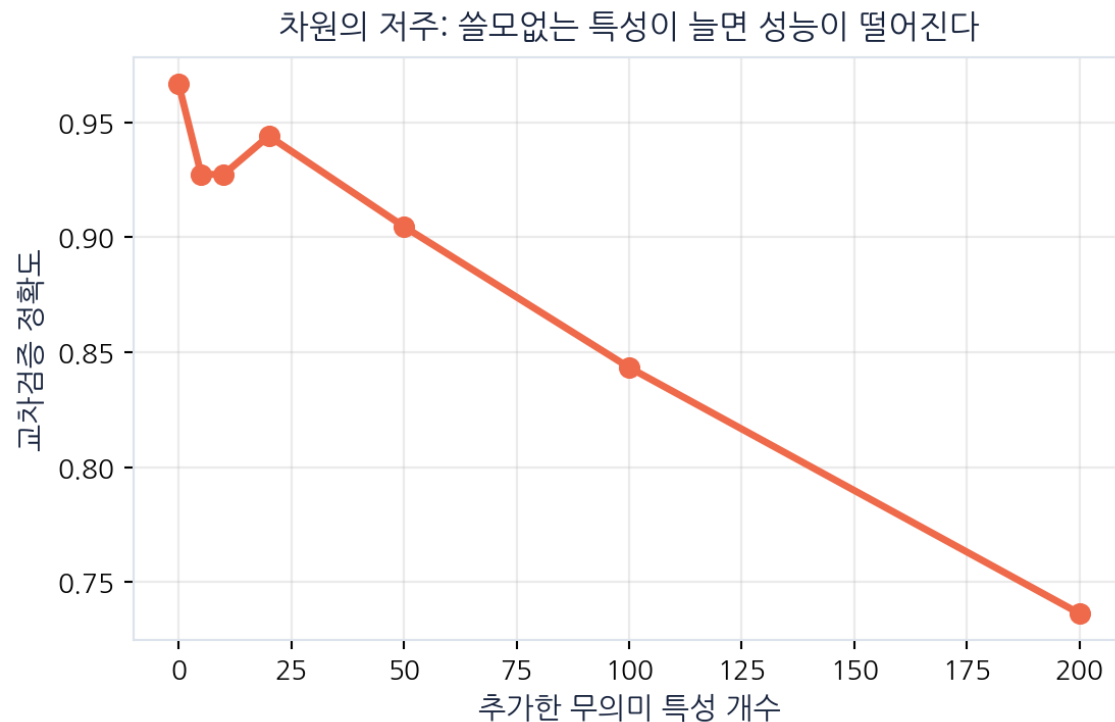
- euclidean : 직선 거리 (기본값)
- manhattan : 격자로 이동한 거리
- 데이터 성격에 따라 유리한 쪽이 다르다

weights — 이웃의 표 크기

- uniform : 이웃 k명 모두 1표씩 (기본값)
- distance : 가까울수록 큰 표
- distance는 k가 커도 성능이 잘 안 떨어진다

③ 차원의 저주 — 특성이 많으면 좋은 것 아닌가?

의미 없는 특성을 늘려가며 직접 확인



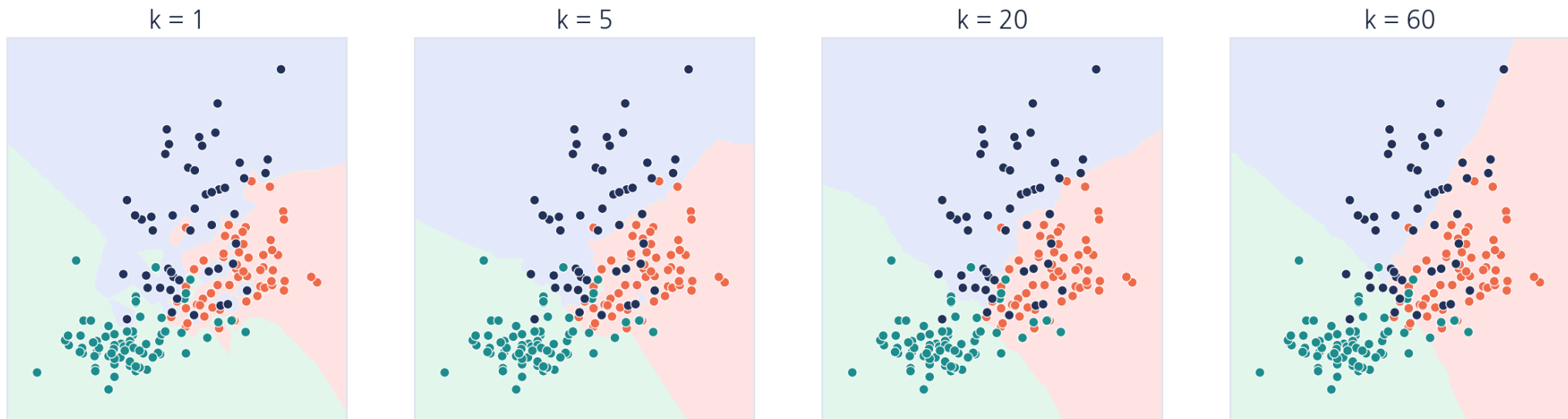
실험 내용

- wine에 '무의미한' 랜덤 특성 추가
- 5개 → 200개까지 늘려가며 측정
- 성능이 계속 떨어진다
- 고차원에서는 모든 점이 비슷하게 떨어져
- '가깝다'는 개념 자체가 흐려진다

③ 결정 경계 — k 가 경계 모양을 결정한다

과대적합·과소적합이 그림으로 보인다

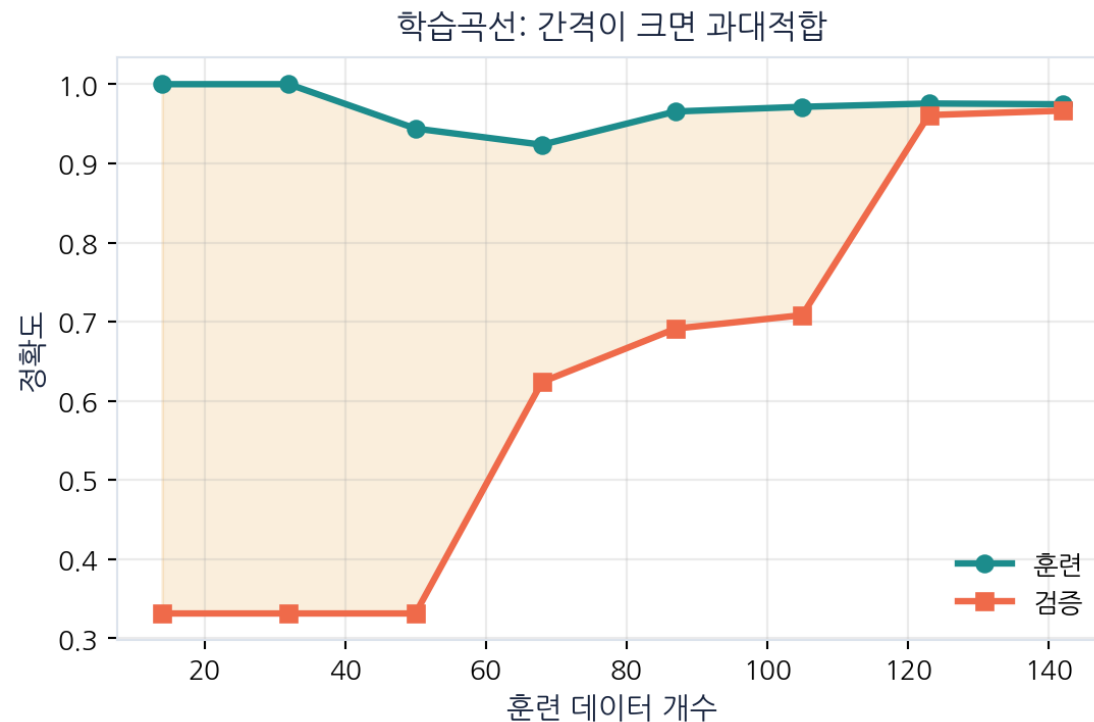
k 가 작으면 복잡한 경계(과대적합), 크면 단순한 경계(과소적합)



$k=1$ 은 점 하나에도 반응해 들쭉날쭉(과대적합) · $k=60$ 은 뭉개져서 지나치게 단순(과소적합)

④ 학습곡선 — '데이터를 더 모아야 할까?'

다음 행동을 정하는 진단 도구



세 가지 진단

- 두 선의 간격이 크다 → 과대적합
(데이터를 더 모으면 도움)
- 낮은 곳에서 붙어 있다 → 과소적합
(모델을 더 복잡하게)
- 검증선이 평평 → 더 모아도 소용없음

심화 정리 — 기억할 것

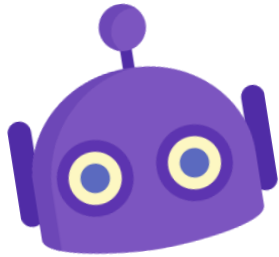
이제 노트북으로 직접 확인합니다

핵심 정리

- 스케일링 효과는 데이터마다 다르다
- 단위 차이가 크면 필수 (wine: +0.28)
- KNN 손잡이는 $k \cdot \text{metric} \cdot \text{weights}$ 셋
- 쓸모없는 특성은 오히려 해롭다
- 학습곡선으로 다음 행동을 정한다

이제 실습으로

- 심화① 스케일러 3종 비교
- 심화② KNN 거리와 가중치
- 심화③ 차원의 저주·결정 경계
- 심화④ KNN 회귀·학습곡선



Decision Tree 의사결정나무 모델 심화

심화 학습

결정 트리를 더 깊이 — 시각화·사후 가지치기·인코딩·회귀 트리

심화 학습

결정 트리를 더 깊이 — 시각화·사후 가지치기·인코딩·회귀 트리

심화 학습에서 다룰 2가지

기본 실습을 마친 뒤 진행합니다

이미 배운 것

- 트리 = 스무고개 (예/아니오로 나눔)
- 불순도가 낮아지는 질문을 고른다
- max_depth 등으로 미리 가지치기
- 범주형은 인코딩이 필요하다

오늘 더 깊이

- ① 다 키운 뒤 잘라내는 '사후 가지치기'
- ② 인코딩이 모델마다 다르게 작용한다

① 가지치기에는 두 가지 시점이 있다

사전 가지치기 vs 사후 가지치기

사전 가지치기 (이미 배움)

- 자라기 전에 미리 제한한다
- max_depth, min_samples_leaf 등
- 간단하지만 '얼마로 정할지'가 감각의 영역

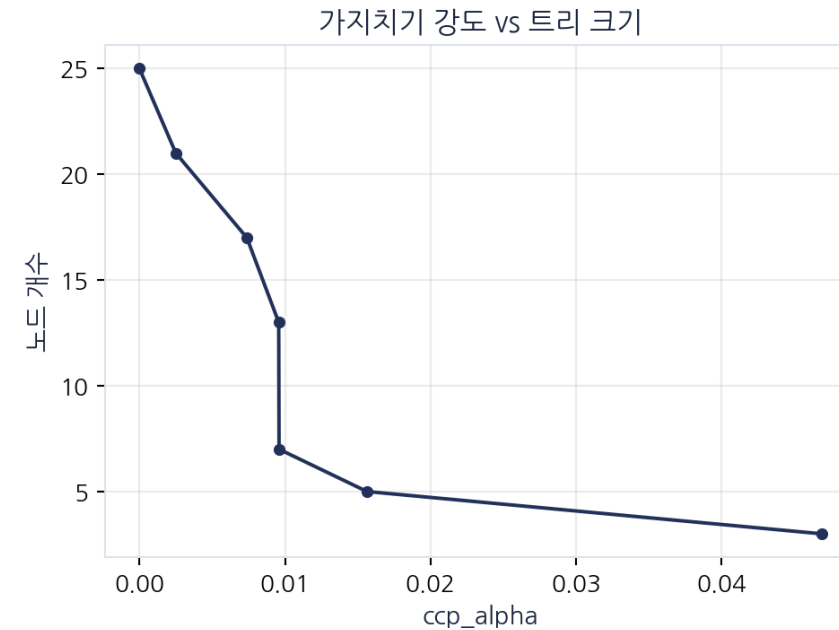
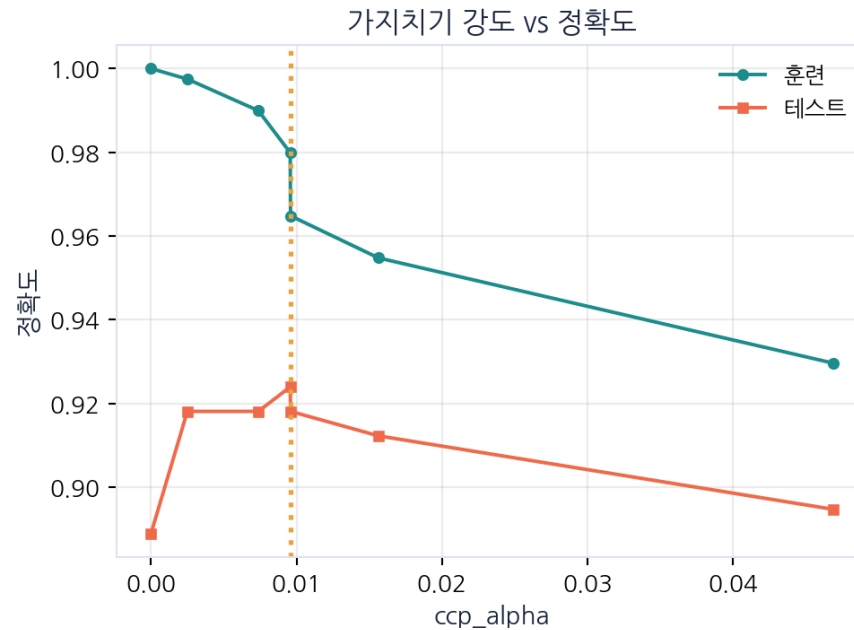
사후 가지치기 (새로 배움)

- 일단 끝까지 키운 뒤 잘라낸다
- ccp_alpha 하나로 조절
- 비용 대비 효과 없는 가지를 자동 제거

비유: 나무를 심을 때 크기를 제한하는 것(사전) vs 다 자란 뒤 전지가위로 다듬는 것(사후)

① ccp_alpha를 키우면 어떻게 되나

가지치기 강도와 성능·트리 크기의 관계



왼쪽: α 가 커질수록 훈련 정확도는 떨어지지만 테스트는 유지·개선 . 오른쪽: 트리 크기(노드 수)가 급격히 줄어든다

ccp_alpha: cost complexity pruning path $\rightarrow$ leaf 하나 당의 몸값 계산

① 코드 — 후보 alpha를 자동으로 얻는다

직접 값을 정할 필요가 없다

```
# 1) 시도할 alpha 후보를 트리가 알려준다
path = tree.cost_complexity_pruning_path(
    X_train, y_train)
alphas = path.ccp_alphas[:-1]

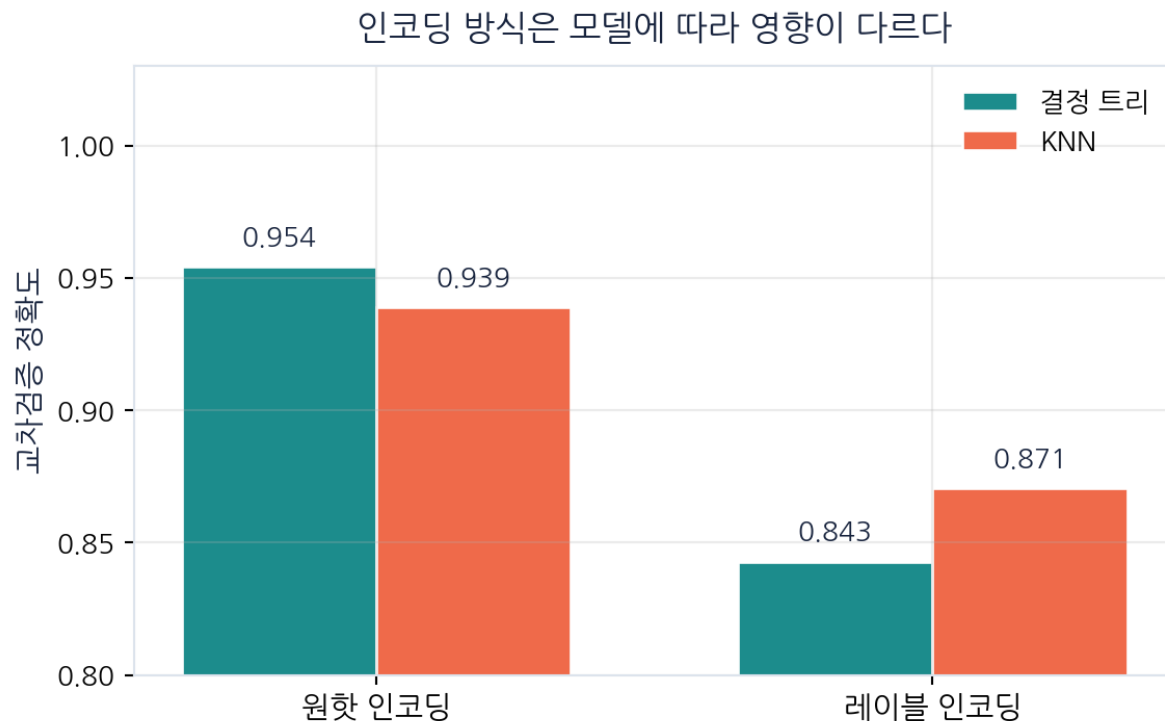
# 2) 각 alpha로 학습해 최적을 찾는다
for a in alphas:
    m = DecisionTreeClassifier(ccp_alpha=a)
    m.fit(X_train, y_train)
```

핵심

- alpha 후보를 직접 정할 필요 없음
- $\alpha \uparrow$ = 많이 잘라냄 = 단순한 트리
- 테스트 성능이 최고인 alpha 선택
- 사전+사후를 함께 써도 된다

② 인코딩 — 모델에 따라 영향이 다르다

같은 인코딩인데 트리와 KNN의 결과가 다르다



왜 이런 차이가?

- 트리 : '이 값보다 큰가?'로 나눈다
→ 숫자를 어떻게 매겨도
여러 번 쪼개면 구분 가능
- KNN : 값의 '거리'를 잰다
→ 가짜 순서가 거리를 왜곡
→ 거리·선형 모델은 원함이 안전

② 인코딩 선택 기준 정리

무엇을 언제 쓸 것인가

인코딩	컬럼 수	가짜 순서	적합한 경우
원핫	많아짐	없음	거리·선형 모델, 순서 없는 범주
레이블	그대로	생김	트리 계열, 범주가 아주 많을 때
순서형	그대로	의도적 사용	실제 순서가 있는 범주 (저·중·고)

실제 순서가 있는 범주(S/M/L, 1·2·3등석)라면 순서형 인코딩이 오히려 정답 — 정보를 살리는 것

심화 정리 — 기억할 것

이제 노트북으로 직접 확인합니다

핵심 정리

- 가지치기는 사전·사후 두 가지
- ccp_alpha로 자동 가지치기 가능
- 인코딩 선택은 모델에 따라 다르다

이제 실습으로

- 심화① 지니vs엔트로피·사후 가지치기
- 심화② 인코딩 3종 비교